

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I**



**BÁO CÁO HÀNG TUẦN MÔN
THỰC TẬP CƠ SỞ**

Giảng viên hướng dẫn	: TS. Kim Ngọc Bách
Sinh viên thực hiện	: Mã sinh viên
Nguyễn Việt Quang	: B23DCCN691

Hà Nội – 2026

1. Mở Đầu

Tuần này tập trung vào hoàn thiện sản phẩm demo cuối cùng thông qua ba hạng mục chính. Thứ nhất, sửa ba lỗi gốc trong file `demo.py` liên quan đến xử lý tài nguyên và xử lý lỗi. Thứ hai, bổ sung tính năng mới cho phép chuyển đổi giữa webcam và video file. Thứ ba, cải thiện tính năng xử lý kích thước frame động để tự thích ứng với nhiều loại camera và độ phân giải khác nhau. Bên cạnh đó, tuần này cũng hoàn thành đánh giá lại phần quantization từ tuần trước, rút ra kết luận rõ ràng về tính khả thi của các phương pháp tối ưu hóa này.

2. Mục Tiêu Thực Hiện

Phần thứ nhất là sửa ba lỗi gốc trong xử lý tài nguyên và xử lý ngoại lệ của file `demo.py`. Các lỗi này tiềm ẩn rủi ro không đóng sạch tệp tin hoặc gây xung đột khi có nhiều thread chạy đồng thời, cần được khắc phục để đảm bảo tính ổn định của ứng dụng. Phần thứ hai là bổ sung tính năng phím V (Video) cho phép người dùng chuyển đổi linh hoạt giữa webcam thực tế và một video file mẫu. Điều này mở rộng khả năng demo ứng dụng, cho phép thử nghiệm trên các scenario được kiểm soát hoặc quay sẵn.

Phần thứ ba là cải tiến cơ chế xử lý kích thước frame động. Thay vì sử dụng kích thước cố định từ frame đầu tiên, hệ thống sẽ tự động phát hiện và thích ứng với kích thước thực tế của mỗi frame, đảm bảo tính tương thích với các camera có độ phân giải khác nhau.

Phần thứ tư là rà soát lại kết quả quantization từ tuần trước, phân tích các nguyên nhân dẫn đến kết quả không như kỳ vọng, và đưa ra kết luận về tính khả thi của các phương pháp nén mô hình trong bối cảnh của dự án.

3. Môi Trường và Thiết Lập Thực Hiện

Quá trình phát triển và thử nghiệm được thực hiện trên máy tính cá nhân với hệ điều hành Windows 11, sử dụng ngôn ngữ Python 3.11. Các thư viện chính bao gồm OpenCV, PyTorch, Transformers của Hugging Face, PIL (Pillow) và pygame. Mô hình BLIP v3 được sử dụng từ thư mục `saved\_models\_v3` từ các tuần trước, tích hợp với VieNeu-TTS để phát giọng nói. Đối với quá trình đánh giá lại quantization, các dữ liệu benchmark từ tuần 23-05 được tái sử dụng.

4. Nội Dung Thực Hiện Chi Tiết

4.1. Fix Ba Lỗi Gốc Trong `demo.py`

4.1.1. Lỗi Thứ Nhất: File .wav Tạm Không Bị Xóa Nếu Xảy Ra Lỗi

Vấn đề:

Trong hàm `speak\_caption()`, file `.wav` tạm thời được tạo tại `tmp\_path`, nhưng nếu lỗi xảy ra giữa các bước pygame (init, load, play, quit), hàm sẽ thoát sớm mà không xóa file. Khi người dùng nhấn Space nhiều lần, hệ thống sẽ để lại hàng chục file .wav orphan trong thư mục tạm, tiêu tốn không gian đĩa.

Giải pháp:

Bọc toàn bộ phần phát âm thanh (từ `pygame.mixer.init()` đến `pygame.mixer.quit()`) vào khối `try/finally`. Phần xóa file được đặt trong `finally` với kiểm tra `os.path.exists(tmp\_path)` trước khi gọi `os.unlink()`, đảm bảo file luôn bị xóa dù có lỗi xảy ra.

4.1.2. Lỗi Thứ Hai: Xung Đột Đa Thread Trên pygame.mixer

Vấn đề:

Hàm `\_run()` bên trong `speak\_caption()` gọi lại `pygame.mixer.init()` và `pygame.mixer.quit()` mỗi lần phát âm thanh. Nếu người dùng nhấn Space liên tục nhanh, hai thread TTS có thể cố gắng khởi tạo hoặc đóng mixer cùng một lúc, dẫn đến race condition hoặc deadlock.

Giải pháp:

Chuyển `pygame.mixer.init()` vào cuối hàm `get\_tts\_engine()`, sao cho mixer chỉ được khởi tạo một lần duy nhất khi TTS engine được tải. Loại bỏ lệnh `pygame.mixer.init()` ra khỏi `\_run()`. Thêm `pygame.mixer.quit()` vào phần cleanup cuối hàm `main()` trước khi chương trình thoát.

Lợi ích:

Tránh khởi tạo lại mixer mỗi lần phát âm thanh

Giảm overhead

Loại bỏ race condition giữa các thread

4.1.3. Lỗi Thứ Ba: Chuyển Webcam Thất Bại → Vòng Lặp Cảnh Báo Vô Tận

Vấn đề:

Khi người dùng nhấn phím C để chuyển sang webcam tiếp theo, nếu `cap.isOpened()` trả về False, hàm `trigger_capture()` vẫn bị gọi trong vòng lặp chính, dẫn đến liên tục in `'[WARN] Không đọc được frame'` mà không có cách thoát ra khỏi tình huống này.

Giải pháp:

Khi người dùng nhấn C, thực hiện fallback loop: thử mở lần lượt các camera còn lại trong `cam_indices`, tối đa `len(cam_indices) - 1` lần. Nếu tất cả đều thất bại, quay lại camera cũ (trước khi thay đổi) và in thông báo rõ ràng. Bọc toàn bộ quá trình trong `try/except` để bảo vệ khỏi các lỗi ngoài dự kiến.

4.2. Bổ Sung Tính Năng Phím V — Video Mode

Cho phép người dùng chuyển đổi giữa webcam và một video file mẫu (ví dụ `'sample_video.mp4'`). Điều này hữu ích khi muốn demo ứng dụng với scenario được kiểm soát hoặc quay sẵn, mà không cần camera thực tế.

Cơ chế hoạt động:

Thêm hằng số `VIDEO_PATH = "sample_video.mp4"` ở đầu file

Nhấn phím V: toggle giữa chế độ webcam (`cam_mode = True`) và chế độ video (`cam_mode = False`)

Khi chuyển từ webcam sang video: mở video file trước, chỉ release camera sau khi mở video thành công → tránh khoảng trống `cap = None`

Video tự động loop lại từ đầu khi hết file (`cv2.CAP_PROP_POS_FRAMES = 0`)

Nếu mở video thất bại, vẫn giữ nguyên source cũ (webcam hoặc video đang chạy)

Lợi ích:

Tăng tính linh hoạt của demo

Cho phép test trên video được kiểm soát

Không cần camera khi muốn demo

4.3. Cải Tiến Xử Lý Kích Thước Frame Động

Vấn đề trước:

Hệ thống sử dụng `cam_h, cam_w` cố định, được xác định một lần từ frame đầu tiên. Nếu người dùng đổi camera có độ phân giải khác hoặc chuyển sang video có aspect ratio khác, các frame mới sẽ bị resize cứng hoặc gây lỗi dimension mismatch khi `np.hstack()`.

Giải pháp:

Bỏ đi các biến `cam_h`, `cam_w` cố định

Thay thế bằng `current_h`, `current_w` được cập nhật động từ `frame.shape[:2]` trong mỗi vòng lặp

Khi người dùng nhấn C (đổi cam) hoặc V (chuyển video), tạo lại `result_panel` với kích thước mới

Khi ở chế độ video: sử dụng `cv2.resize()` với letterbox method (giữ tỉ lệ gốc, nền đen) để resize về đúng kích thước khung trước khi hiển thị và đưa vào model

Lợi ích:

Tự động thích ứng với bất kỳ camera hoặc video nào

Loại bỏ risk dimension mismatch

Tăng tính robust của ứng dụng

4.4. Đánh Giá Lại Phần Quantization

Tuần 23-05 thực hiện quantization mô hình BLIP với PyTorch Dynamic Quantization (INT8) và chuẩn bị pipeline ONNX. Kết quả cho thấy PyTorch INT8 chậm hơn FP32 đáng kể (63.4% latency tăng) và chất lượng giảm (BLEU-4 từ 24.50 xuống 20.29, giảm 4.21 điểm). Tuần này tiến hành rà soát toàn bộ kết quả.

Model	Precision	BLEU-4	ROUGE-L	Latency (s)	Status
baseline_fp32_greedy	FP32	22.83	49.85	17.80	Baseline
onnx_fp32	FP32	22.83	49.85	14.01	1.27x speedup
pytorch_int8	INT8	15.55	46.21	26.05	Chậm hơn 46%

Nguyên nhân thất bại của PyTorch INT8:

Dynamic Quantization tính toán scale factor + zero-point trong thời gian inference, phát sinh overhead ở mỗi bước sinh token

CPU không có tập lệnh tối ưu cho INT8 (VNNI trên Intel Cascade Lake)

Batch size 16 quá nhỏ để khai thác hiệu quả phép nhân nguyên song song

Model sinh chuỗi (autoregressive) là worst-case scenario cho quantization vì mỗi bước xử lý chỉ 1 token, chi phí quantize/dequantize lớn hơn lợi ích

Khám phá bất ngờ:

ONNX FP32 (không quantize) đã đạt 1.27x speedup so với PyTorch FP32 greedy mà chất lượng caption không suy giảm. Điều này cho thấy ONNX Runtime có khả năng tối ưu hóa đồ thị tính toán (graph optimization, operator fusion) hiệu quả, loại bỏ nhu cầu quantization.

Kết luận:

Quantization không phải lúc nào cũng giúp ích. Hiệu quả phụ thuộc mạnh vào backend (PyTorch vs ONNX), kiến trúc model (CNN vs Transformer), bài toán (classification vs sequence generation), và phân cứng target. Trong bối cảnh dự án, ONNX FP32 là giải pháp tối ưu nhất.

5. Nhận Xét và Đánh Giá

5.1. Về Ba Lỗi Gốc Đã Sửa

Ba lỗi đã sửa là những vấn đề tiềm ẩn có thể dẫn đến hệ thống không ổn định. Việc xử lý try/finally cho file .wav là phần quan trọng để đảm bảo dọn dẹp tài nguyên. Việc khởi tạo pygame.mixer một lần duy nhất loại bỏ race condition tiềm ẩn khi có nhiều thread chạy cùng lúc. Việc thêm fallback loop cho chuyển webcam đảm bảo ứng dụng không bị "treo" ở trạng thái không khả dụng.

5.2. Về Tính Năng Video Mode

Tính năng phím V cho phép linh hoạt hóa demo, mở ra khả năng test trên video được kiểm soát. Cơ chế letterbox resize đảm bảo video mặc dù có aspect ratio khác nhau vẫn có thể xử lý được mà không mất thông tin hay gây lỗi.

5.3. Về Xử Lý Kích Thước Frame Động

Việc chuyển sang xử lý kích thước động là một sự cải thiện quan trọng về mặt robust. Ứng dụng hiện tại có thể hoạt động với bất kỳ camera hoặc video nào mà không cần cấu hình trước. Letterbox resize là kỹ thuật phổ biến trong object detection, đảm bảo không mất thông tin spatial khi resize ảnh có aspect ratio khác nhau.

5.4. Về Quantization

Phản đánh giá lại quantization rút ra một bài học quan trọng: không phải tất cả optimization techniques đều áp dụng được cho mọi bài toán. PyTorch Dynamic Quantization thất bại trong trường hợp này, nhưng ONNX FP32 lại thành công. Báo cáo chi tiết về lý do thất bại của quantization có giá trị khoa học cao hơn so với việc che giấu hoặc bỏ qua vấn đề, cho thấy thái độ nghiên cứu chuyên nghiệp.

6. Kết Luận

Trong tuần này, em đã hoàn thiện sản phẩm demo `demo.py` bằng cách sửa ba lỗi gốc liên quan đến xử lý tài nguyên và xử lý ngoại lệ, bổ sung tính năng chuyển đổi giữa webcam và video file thông qua phím V, cải tiến cơ chế xử lý kích thước frame động để tự thích ứng với mọi loại camera. Đồng thời, tuần này cũng hoàn thành đánh giá lại phần quantization từ tuần trước, kết luận rằng quantization không phù hợp với kiến trúc model sinh chuỗi, nhưng ONNX FP32 đã đạt được speedup 1.27x mà không cần quantization. Sản phẩm demo hiện tại đã sẵn sàng cho demo cuối kỳ, với tính ổn định cao, khả năng thích ứng linh hoạt, và xử lý lỗi toàn diện. Trong giai đoạn tiếp theo, em sẽ tập trung vào việc viết báo cáo tổng kết cuối kỳ, chuẩn bị slide demo, và tổng hợp toàn bộ mã nguồn lên GitHub theo đúng cấu trúc yêu cầu của học phần.